Bankai: Ultra-Sparse Adaptation of 1-Bit LLMs via XOR Patches

Nikshep Saravanan
github.com/nikshepsvn/bankai

April 2, 2026

Abstract

True 1-bit large language models have no post-training adaptation method. LoRA, fine-tuning, and quantization-aware training all require continuous weights or gradients that binary models lack. We introduce **Bankai**, the first post-training adaptation method for true 1-bit LLMs, using bitwise XOR operations on binary weights. Bankai patches are sparse XOR bitmasks that modify model weights in-place with a single bitwise operation, incur zero inference overhead, and compress to around one kilobyte.

We validate on Bonsai 8B (PrismML, 2026), a true 1-bit, 8.2 billion parameter language model. Through eleven experiments, we demonstrate that: (1) binary MLP weights exhibit massive redundancy; (2) scale-guided bit flips produce 3.88× more behavioral impact than random flips; (3–4) greedy search yields patches that correct specific calculus failures in free generation; (5) patches trained on few probes memorize rather than generalize; and (6) training on diverse probe variations produces patches that **generalize to held-out prompts**—fixing 4 of 17 problems the base model gets wrong (23.5%) with zero breakage on the 13 it already solves.

Correction to this version. The Experiment 6 figure quoted above (4 of 17, 23.5%) is superseded. An audit prompted by an independent reproduction found three defects in the probe measurement; under corrected scoring the same patch fixes **2** held-out problems with zero breakage, confirmed at the generation level rather than by logit gaps. The central claim is unaffected. See Section 8.

1 Introduction

The emergence of true 1-bit large language models—where every weight is a single bit—represents a fundamental shift in model efficiency. Bonsai 8B [PrismML, 2026] stores 8.2 billion parameters in 1.15 GB, runs at 44 tokens/second on an iPhone, and achieves benchmark scores competitive with full-precision 8B models.

However, these models present a novel challenge: **they cannot be adapted after deployment**. Every existing method for post-training adaptation requires continuous-valued operations:

- **LoRA** [Hu et al., 2021] adds low-rank weight deltas in float space.
- **Fine-tuning** backpropagates continuous gradients through differentiable weights.
- **Quantization-aware training** adjusts weights during pre-training, not after deployment.

None of these apply to a model whose weights are single bits. Once a 1-bit model is deployed, it is frozen.

We introduce **Bankai**, the first post-training adaptation method for true 1-bit LLMs. The key insight: because weights are bits, the "diff" between two behavioral states is a bitwise XOR mask. Where the mask has a 1, a weight flipped. Where it has a 0, nothing changed.

For nearby behavioral variants, this mask is sparse—enabling patches that modify 0.007% of weights (∼1 KB) with zero inference overhead, microsecond application, and exact reversibility.

We validate Bankai on Bonsai 8B through six experiments, progressing from mechanism validation (Experiments 1–2), through targeted correction (Experiments 3–4), to generalization testing (Experiments 5–6). We find that patches trained on 6 probes memorize specific prompts, but patches trained on 60 diverse probes generalize to held-out problems—fixing 4 of 17 incorrect answers with zero collateral damage.

1.1 Contributions

1. **The first post-training adaptation method for true 1-bit LLMs.** Existing methods (LoRA, fine-tuning, QAT) require continuous weights or gradients and are inapplicable to binary architectures.
2. **Scale-guided targeting.** Per-group FP16 scale factors predict which weight regions are most behaviorally sensitive, enabling 3.88× more efficient search than uniform random sampling.
3. **Layer-level functional specialization in 1-bit architectures.** Early layers (1–4) and the penultimate layer (34) are load-bearing, while middle layers (17–21) contribute minimally.
4. **A characterization of generalization.** Patches trained on few probes memorize; diverse training produces generalization. This is the first empirical characterization of how XOR patches learn.
5. **An open toolkit and patch format** for reproducible creation, application, and evaluation of XOR patches.

2 Background

2.1 True 1-Bit vs. Ternary Models

The distinction between true 1-bit and ternary models is critical. BitNet b1.58 [Wang et al., 2023, Ma et al., 2025] uses ternary weights $\{-1, 0, +1\}$, requiring 2 bits per weight in storage. Despite the "1-bit" branding, XOR on 2-bit ternary encodings produces invalid states: $\text{XOR}(01, 10) = 11$, which has no valid ternary mapping.

Bonsai 8B [PrismML, 2026] is a true 1-bit model where each weight is a single bit: $0 \to -\text{scale}$, $1 \to +\text{scale}$. Weights are packed as `uint32` arrays with per-group FP16 scale factors (group size 128). This makes bitwise XOR a semantically valid operation. As of April 2026, Bonsai 8B is the only production-quality true 1-bit LLM.

2.2 Deployment Implications

The properties of XOR patches—kilobyte-scale size, microsecond application, zero inference overhead, instant reversibility—enable a deployment model impossible with existing adaptation methods. A library of domain patches (math, code, medical, legal), each ∼1 KB, can be hot-swapped at inference time on a phone. The base model stays at 1.15 GB; a thousand patches add 1 MB. LoRA adapters are ∼100 MB each, add per-token latency, and require weight reloading to swap.

3 Related Work

Low-rank adaptation. LoRA [Hu et al., 2021] and its variants (QLoRA [Dettmers et al., 2023]; DoRA) reduce fine-tuning cost via low-rank weight deltas. These require continuous weights and are not applicable to true 1-bit architectures.

Compact adaptation. RECAST [Xu et al., 2024] reduces task-specific parameters to fewer than 50 via weight reconstruction on continuous-weight models.

Binary neural networks. XOR-Net [Bulat and Tzimiropoulos, 2020] and XNOR-Net [Rastegari et al., 2016] use binary operations for efficient forward passes, not post-training behavioral modification.

1-bit and sub-1-bit LLMs. BitNet [Wang et al., 2023] introduced 1.58-bit LLM training. STBLLM [Dong et al., 2024] pushed compression below 1-bit and observed that some weights in binarized LLMs can be randomly flipped without significant performance degradation—a finding our Experiment 1 independently confirms.

Bit-flip attacks. Rakin et al. [2019] showed that ∼20 targeted bit flips can catastrophically degrade a quantized DNN. Subsequent work (T-BFA [Bai et al., 2021]; Versatile Weight Attack) refined targeted attacks. This literature establishes that small numbers of bit flips produce outsized behavioral effects—but focuses exclusively on adversarial degradation. **Bankai inverts this mechanism:** constructive bit flips for targeted capability improvement.

Model editing. ROME [Meng et al., 2022] and MEMIT [Meng et al., 2023] perform targeted factual edits on continuous-weight models. Bankai shares the goal of minimal intervention but operates on binary weights for behavioral (not factual) modification.

4 Methodology

All experiments use Bonsai 8B (`prism-ml/Bonsai-8B-mlx-1bit`), a true 1-bit, 8.2B parameter model based on the Qwen3 architecture (36 layers, 4096 hidden dim, GQA with 32/8 heads). Weights are packed as `uint32` arrays with 1-bit group quantization (`group_size=128`). Experiments were run on Apple M3 (24 GB).

4.1 Probe Metrics

Experiments 1–11 use the `token_gap` metric described first; it has defects documented in Section 8. Experiment 12 uses `seq_logprob` for search and greedy generation for reporting.

`token_gap` (Experiments 1–11). Pairs of (correct_token, wrong_token) following a deterministic prompt. The logit gap $G = \text{logit}(\text{correct}) - \text{logit}(\text{wrong})$ is a single-forward-pass measurement: positive means the model prefers the correct answer. Answer strings are reduced to a single token id by keeping their last subtoken. We used this as a fast proxy for behavioral change, noting that logit improvement does not always translate to generation improvement (see Section 9).

`seq_logprob` (Experiment 12). The gap is the difference in summed teacher-forced log-probability between the full correct answer string and a plausible per-probe distractor, with continuations tokenized in the context of their prompt. Multi-token answers, leading spaces, and same-suffix collisions are all handled correctly. This is used as search fitness only.

Greedy generation (Experiment 12, headline). What the model actually emits under greedy decoding, compared against the expected answer and any accepted alternate renderings. It depends on no distractor token, so it cannot be satisfied by shifting probability between two hardcoded ids. Logit gaps are a search signal; generation is the result.

4.2 Backends: MLX and GGUF/CUDA

Bankai has two independent backend implementations:

- **MLX** (Apple Silicon): uses PrismML's MLX fork with 1-bit kernels. Implements `flip_row` as a bitwise XOR on packed `uint32` weight tensors, followed by `mx.eval()` to propagate the modification.
- **GGUF/CUDA** (NVIDIA GPUs): uses a custom C++ tool, `bankai_eval.cpp`, built against PrismML's llama.cpp fork. The tool loads Bonsai 8B once and exposes a line protocol on stdin/stdout (`PROBE`, `TOKENIZE`, `FLIP_ROW`, `FLIP_GROUP`, `SCALES`, `NUM_ROWS`). Weight modifications happen entirely in GPU memory via `ggml_backend_tensor_get/set` on the Q1_0_g128 tensor data — no file reloading, no subprocess restart. The Python `GGUFBackend` wraps this subprocess and pipelines probe commands through the stdin buffer, so a full multi-probe iteration costs one write, one flush, and one batched read. This achieves approximately 24× speedup over the MLX path on Apple M3: a 300-iteration search runs in ∼3 min on Modal L40S vs ∼67 min on M3 MacBook Air.

Patch files (JSON lists of `(layer, proj, row[, group])` tuples) are portable between the two backends as long as the tensor names match. The MLX and GGUF versions of Bonsai differ in quantization format, however (MLX uses scale + bias per 128-weight group at 1.25 bpw; GGUF uses scale-only at 1.125 bpw), so identical patches can produce different logit gaps on the two formats — this is discussed in Experiment 9.

4.3 Experiment 1: Random Bit Flip Robustness

We flip N random bits across all MLP weight tensors (5.4B bits total) and measure perplexity on a fixed 5-sentence eval set, for $N \in \{100, 1\text{K}, 10\text{K}, 50\text{K}, 100\text{K}, 500\text{K}\}$ under four strategies: random across all layers, random in layers 16–24, scale-guided medium (25th–75th percentile), and scale-guided high (top 25%).

4.4 Experiment 2: Structured Flips and Layer Specialization

For each of 36 layers, we flip all bits in the entire MLP and measure logit gaps on 8 probes spanning arithmetic, code, geography, and science. We then test row-level flips at varying counts (1–1024 rows) in the most impactful layer and compare high-scale rows vs. random rows at 64 rows.

4.5 Experiment 3: Greedy Patch Search (Arithmetic)

Greedy hill climbing over row-level flips in layers $[1, 2, 3, 4, 34]$ (selected from Experiment 2). Each iteration: sample a row weighted by scale magnitude, flip all 4,096 bits, measure fitness, keep if improved, revert if not. 200 iterations with 6 target probes (arithmetic) and 4 control probes (knowledge).

Fitness function:

$$f = \frac{1}{|T|} \sum_{t \in T} (g_t - g_t^0) - \lambda \cdot \frac{1}{|C|} \sum_{c \in C} \max(0, g_c^0 - g_c) \tag{1}$$

where g_t, g_c are target and control logit gaps, superscript 0 denotes baselines, and $\lambda = 2.0$ penalizes control degradation.

4.6 Experiment 4: Calculus Patch

Same search targeting 6 calculus probes (polynomial differentiation, second derivatives, integrals, primality, trigonometry, exponential derivatives) where the base model fails deterministically (verified across 5 runs). 300 iterations with screening optimization.

4.7 Experiment 5: Variation Testing

We generate 15 novel variations per probe category (90 total) and measure sign flips (wrong→right vs. right→wrong) on the Experiment 4 patch applied to these never-seen probes.

4.8 Experiment 6: Generalization-Optimized Search

Train on 60 probes (10 per category with diverse phrasings), hold out 30 probes (5 per category) for validation. Mean fitness, 300 iterations. Search time scales roughly linearly with probe count: 6 probes → 13 min, 60 probes → 67 min.

4.9 Experiment 7: Patch Stacking

Apply the Experiment 3 math patch (72 flips) and Experiment 4 calculus patch (70 flips) simultaneously via sequential XOR. Verify order independence and reversibility. Test both math and calculus probes.

4.10 Experiment 8: GSM8K Safety Check

Run 50 GSM8K word problems with full generation (400 tokens) and answer extraction. Compare accuracy with and without the Experiment 6 generalized patch to check for collateral damage on general math reasoning.

4.11 Experiment 9: GGUF/CUDA Backend (Beefier Search)

Port Bankai to PrismML's llama.cpp fork (Q1_0_g128 CUDA kernels) via a custom C++ tool, `bankai_eval`, that performs in-GPU weight manipulation through `ggml_backend_tensor_get/set`. Run the Experiment 6 search on Modal L40S with an expanded layer set $[1, 2, 3, 4, 5, 6, 10, 34]$ (adding layers 5, 6, 10 which the Experiment 2 layer-impact map identified as calculus-sensitive) and 800 iterations instead of 300.

4.12 Experiment 10: Attention Projection Search

Same configuration as Experiment 9 but with `search_projs = [gate, up, q, k, v, o]` — the search now considers attention Q/K/V/O projections in addition to MLP. First exploration of attention-projection XOR flips on a 1-bit LLM.

4.13 Experiment 11: Per-Group Granularity Search

Replace row-level flipping with group-level: each "flip" XORs only the 128 sign bits of one 18-byte Q1_0_g128 block instead of all 4,096 sign bits of a full row. Search space grows 32× (from ∼131K rows to ∼6.3M groups). 1500 iterations, same 8 layers and 2 MLP projections.

5 Results

5.1 Experiment 1: Random Flips Are Absorbed

The model is remarkably robust to random perturbation in MLP weights, implying massive redundancy. (Attention weights, embeddings, and the LM head were not tested.)

5.2 Experiment 2: Layer Impact and Scale-Guided Targeting

Scale-guided targeting: At 64 rows in layer 34, high-scale rows produce 3.88× more behavioral impact than random rows (avg $|\Delta\text{gap}|$: 0.459 vs. 0.118). Layer effects appear domain-specific across our probe set—layer 34 shifts physics by +13.8 while shifting code by −15.9.

Table 1: Perplexity change under random bit flips in MLP weights.

Flips	% of MLP bits	Max Perplexity Δ
100	0.000002%	< 0.01
10,000	0.0002%	< 0.01
100,000	0.002%	< 0.02
500,000	0.009%	< 0.08

Table 2: Average absolute logit gap change ($|\Delta\text{gap}|$) when flipping the entire MLP of each layer range. 8 probes across 4 domains.

Layer range	Avg $|\Delta\text{gap}|$	Interpretation
0–4 (early)	3.2–7.2	High impact
5–16 (middle)	0.7–3.0	Moderate, decreasing
17–21 (deep mid)	0.7–1.6	**Lowest impact**
22–33 (late)	1.6–3.4	Moderate, increasing
34 (penultimate)	**9.0**	**Highest impact**
35 (final)	3.2	High

5.3 Experiment 3: Arithmetic Patch

200 iterations, 72 accepted flips, 7.5 minutes on M3. The `2+2` probe flips from incorrect (prefers 5) to correct (prefers 4) in both logit gap and free generation. The `7×8` probe improves at the logit level (prefers 56 over 54) but free generation produces 64 ($= 8 \times 8$)—a different wrong answer, illustrating the gap between probe-based and generation-based evaluation.

Patch: 72 flips, 294,912 bits modified (0.005%), 864 bytes. No degradation on 4 measured control probes.

5.4 Experiment 4: Calculus Patch

300 iterations, 70 accepted flips, ∼13 minutes on M3. Two deterministic failures corrected in free generation:

- `d/dx [x^4 + 3x^2] = 0` → `4x^3 + 6x` ✓
- `The second derivative of x^4 is 12x^3` → `12x^2` ✓

Prompt sensitivity finding: The primality probe (`Is 97 prime?`) is sensitive to a trailing space—the base model's answer flips from "No" to "Yes" without any patch. This illustrates the fragility of single-token probes and motivates diverse training sets (Experiment 6).

Patch: 70 flips, 286,720 bits (0.005%), 840 bytes. No degradation on 5 measured control probes.

5.5 Experiment 5: Variation Testing (Negative Result)

Superseded—and understated. 7 of these 90 variation probes are dead under `token_gap` (Section 8). Rerun under corrected scoring (Section 7), this patch breaks **15** probes rather than 5, taking accuracy from 49/90 to 36/90. The conclusion below is correct and in fact too generous. Numbers left unmodified as the published record.

The Experiment 4 patch was tested on 90 novel variations (15 per category). Sign-flip analysis:

Finding: The 6-probe patch memorizes specific prompts rather than shifting capabilities. All 5 broken probes had borderline baseline gaps (< 1.2).

Table 3: Experiment 4 patch applied to 90 never-seen probes.

Metric	Count
Fixed (wrong → right)	2
Broke (right → wrong)	5
Stayed right	57
Stayed wrong	26
Net sign flips	−3
Base accuracy	62/90 (68.9%)
Patched accuracy	59/90 (65.6%)

5.6 Experiment 6: Generalization-Optimized Search

Superseded. The fix counts in this subsection are inflated by probe measurement defects (Section 8). Under corrected scoring the same patch fixes **2** held-out problems, not 4, with zero breakage; three of the four "fixes" were already correct at baseline. Numbers are left unmodified as the published record.

Motivated by Experiment 5's memorization finding, we trained on 60 probes (10 per category with diverse phrasings) and validated on 30 held-out probes.

Table 4: Sign-flip analysis on training and held-out validation sets.

Metric	Training (60)	Validation (30)
Fixed (wrong → right)	4	**4**
Broke (right → wrong)	0	**0**
Stayed right	44	13
Stayed wrong	12	13
Accuracy	44/60 → 48/60	13/30 → **17/30**

Validation fixes span 4 categories: polynomial derivatives, second derivatives, integrals, and primality. Trig and exponential derivative categories saw zero fixes—a calculus-specific layer impact analysis identified layers 5, 6, and 10 as high-impact for calculus but not included in the search set, which may explain the gap.

The base model gets 17 of 30 validation probes wrong; the patch fixes 4 of those 17 (23.5%) while breaking none of the 13 it already solves.

Patch: 93 flips, 380,928 bits (0.007%), 1,116 bytes. No degradation on 5 measured control probes.

5.7 Experiment 7: Patch Stacking

The Experiment 3 math patch (72 flips) and Experiment 4 calculus patch (70 flips) have zero row overlap, yielding 142 combined flips. Stacking is mechanically correct: order-independent (math+calc gives identical results to calc+math) and perfectly reversible (zero drift after removal).

However, behavioral composition shows interference. The math patch damages `poly_deriv` (−1.59) while the calculus patch improves it (+2.77); stacked, they partially cancel (+0.19). Overall: individual patches each fix 2 and break 1 probe; the stacked patch fixes 1 and breaks 0.

Finding: Patches compose algebraically but interfere behaviorally. Stacking is safer (fewer breakages) but less effective (fewer fixes) than individual patches. Joint optimization—searching for flips that improve both domains simultaneously—would likely outperform naive stacking.

5.8 Experiment 8: GSM8K Safety Check

50 GSM8K word problems, generation with answer extraction:

Table 5: GSM8K accuracy with and without patch (50 problems).

	Without patch	With patch
Correct	11/50	14/50
Accuracy	22.0%	28.0%

No degradation detected. The patch slightly improved accuracy (+3 problems), likely within noise for $n = 50$ but directionally positive. Our GSM8K accuracy (22%) is below Bonsai's reported benchmark (88%) due to evaluation harness differences; the relative comparison is the meaningful signal.

5.9 Experiment 9: GGUF/CUDA Backend

Search: 800 iterations on Modal L40S, 8 layers $[1, 2, 3, 4, 5, 6, 10, 34]$ $\times$ 2 MLP projections. 424 seconds total — approximately 9.5$\times$ faster than the original M3 run (4018 s).

Patch: 73 flips (54 screened out), 876 bytes. Layers 5, 6, and 10 all contributed accepted flips, confirming the Experiment 2 layer-impact map's prediction.

Table 6: Experiment 9 results (Modal L40S, GGUF backend).

Set	Baseline	Patched
Training (60 probes)	43/60	42/60
Validation (30 held-out)	**13/30**	**16/30**
Validation fixed	—	**3**
Validation broke	—	**0**

Finding: The GGUF format (scale-only Q1_0_g128, 1.125 bpw) is strictly less expressive than MLX (scale+bias, 1.25 bpw), and baseline logit gaps differ between formats (for example, Paris-vs-London is +6.50 on MLX vs +3.54 on GGUF for identical tokens). Despite the format difference, the GGUF search with expanded layers recovers 3/4 of the MLX validation result (3 fixed vs 4) while running in a fraction of the time. This establishes the GGUF/CUDA path as a viable production route.

5.10 Experiment 10: Attention Projection Search

Search: Same as Experiment 9 with six projections per layer (gate, up, q, k, v, o). 800 iterations, 433 seconds.

Patch: 85 flips total — 69 MLP (gate: 37, up: 32) and **16 attention** (q: 7, o: 4, k: 3, v: 2). All four attention projection types received accepted flips, confirming the mechanism is mechanically valid on attention weights.

Table 7: Experiment 10 vs Experiment 9 — adding attention to the search.

	Exp 9 (MLP only)	Exp 10 (+ attention)
Training fixed/broke	2 / 3	2 / 1
Validation fixed	**3**	**0**
Validation broke	0	0

Finding (negative result): Attention flips are mechanically functional but **hurt generalization**. Adding the attention search space to Experiment 9 eliminated all three held-out validation fixes. Every probe category that had improved in Experiment 9 regressed to zero fixes in Experiment 10.

Interpretation: Attention weights encode context-specific routing patterns. Flipping rows of attention projections finds modifications that improve training-set fitness but depend on exact prompt structure and don't transfer to novel phrasings. MLP weights, by contrast, encode more abstract representations whose modifications generalize. **For behavioral patching of 1-bit LLMs, MLP is the right target; attention is too context-bound.** This is, to our knowledge, the first empirical characterization of attention XOR flips on a 1-bit model.

5.11 Experiment 11: Per-Group Granularity Search

Search: 1500 iterations, `granularity="group"`, 755 seconds. Candidate space: 6.3 million group-flips (vs ∼131K row-flips in Experiment 9).

Patch: Only 6 flips accepted (of 1500 iterations, ∼0.4% accept rate). 331 screened out. Max fitness reached: +0.0067 (vs +0.32 for the row-level beefy run in Experiment 9 — two orders of magnitude smaller).

Table 8: Experiment 11: per-group flips produce no measurable behavior change.

Set	Baseline	Patched
Training (60 probes)	43/60	43/60
Validation (30 held-out)	13/30	13/30
Fixed / broke	—	0 / 0

Finding (negative result): Per-group flipping is too fine-grained for the current mean-fitness search. Each 128-bit flip produces probe gap changes on the order of 0.001–0.01 — approximately 10× smaller than row-level flips — and the control-degradation penalty ($\lambda = 2.0$) dominates at this scale. The search correctly rejected ∼99.6% of candidates, and the six accepted flips produced no measurable behavioral change.

Interpretation: Finer granularity is not free. It requires either (a) a more sensitive fitness function (log-probability differences instead of logit gaps), (b) a lower control penalty that allows small improvements through, (c) cumulative flipping (accepting pairs or triples of groups together to cross the noise threshold), or (d) a different search algorithm (evolutionary with population-level moves, or gradient-free optimization with variance reduction). Naive greedy hill climbing at group granularity is not viable for this problem.

6 Experiment 12: Corrected Metric Replication

Experiment 12 repeats Experiment 6 with everything held fixed—the same 90 prompts, the same layers $[1, 2, 3, 4, 34]$, the same 300 iterations, the same seed, the same control probes, the same mean fitness—and changes only the measurement. Search fitness becomes `seq_logprob`; the reported metric becomes greedy generation accuracy, which depends on no distractor token.

The probe set keeps the original prompts and repairs the three defects of Section 8: full answer strings rather than single token ids, per-probe distractors reflecting the mistake a model actually makes, and a single answer convention for trigonometry. Probes may declare alternate renderings, so a model writing 0.7071 for $\sqrt{2}/2$ is not marked wrong.

All five fixes are changes in what the model emits: `d/dx [x^7 + x]` from `0` to `7x^6 + 1`; the second derivative of $3x^3$ from `18x^2` to `18x`; `Is 113 prime?` and `Is 53 prime?` both from `No` to `Yes`; and `d/dx [e^(-2x)]` from a malformed `-2e^(-2x) + 0 = -2` to `-2e^(-2x)`. Primality

Table 9: Experiment 12, validation set (30 held-out probes), greedy generation.

Metric	Count
Fixed (wrong → right)	**5**
Broke (right → wrong)	**0**
Accuracy	14/30 → **19/30**
Patch size	78 flips, 936 bytes

improves most (2/5 to 4/5), consistent with it being the one original category whose contrast alternated direction and so could not be won by a token bias. One training-set probe regresses (`int_train_7`) and is reported rather than screened out.

This is a strictly harder bar than Experiment 6's: the model must emit the complete correct expression rather than rank one token above another. The comparison is therefore not $4 \rightarrow 2$ but *4 claimed under a broken metric* versus *5 demonstrated under a sound one.*

6.1 Extended Layer Set: A Negative Result

Experiment 6 speculated that trigonometry and exponential derivatives saw zero fixes because layers 5, 6 and 10—identified as high-impact for calculus—were absent from the search set. Rerunning with $[1, 2, 3, 4, 5, 6, 10, 34]$ and changing nothing else falsifies this.

Table 10: Baseline versus extended layer set, corrected metric, 300 iterations.

	Baseline $[1, 2, 3, 4, 34]$	Extended $+[5, 6, 10]$
Train (generation)	30/60 → 33/60	30/60 → **35/60**
Held-out (generation)	14/30 → **19/30**	14/30 → 17/30
Held-out fixed	**5**	3
Held-out broke	0	0
Patch	78 flips, 936 B	95 flips, 1,140 B

The extended search fits the training set better and generalizes worse. Because training accuracy *improved*, this is overfitting rather than under-sampling of the larger candidate pool. The hypothesis fails on its own terms: trigonometry is unmoved (2/5 to 2/5) and second derivatives are unmoved (0/5 to 0/5).

This replicates the Experiment 10 pattern from an independent direction. There, adding attention projections eliminated all held-out fixes; here, adding MLP layers halves them. Two unrelated expansions of the search space at fixed iteration budget both traded generalization for training fit, suggesting the constraint is a property of greedy hill climbing on this objective rather than of any particular layer or projection type. The narrower search set is the better one, and Experiment 6's layer choice was sound even though its stated explanation for the trig gap was not.

Both runs used 300 iterations while the candidate pool grew from ∼131K to ∼210K rows, so the extended run sampled a smaller fraction of its space. A coverage-matched rerun would separate "more layers hurt" from "more layers need more iterations," though it would not rescue the claim about trigonometry.

7 Experiment 13: Corrected Variation Testing

Experiment 13 repeats Experiment 5 as an evaluation only: the same 6-probe Experiment 4 patch applied to the same 90 novel variation prompts, with the probe set repaired as in Exper-

iment 12 and scored by full-answer log-probability and greedy generation.

Table 11: Experiment 5 as published versus Experiment 13 corrected, 90 variations.

Metric	Experiment 5	Experiment 13
Fixed (wrong → right)	2	2
Broke (right → wrong)	5	**15**
Accuracy	62/90 → 59/90	49/90 → **36/90**

Under `seq_logprob` the picture is the same: 65/90 to 57/90, with 0 fixed and 8 broken. Damage appears in five of six categories.

Finding. Experiment 5's conclusion holds and was understated. A patch trained on 6 probes does not merely fail to generalize; it actively degrades capability. The original writeup rationalized its 5 breaks as "all borderline cases with baseline gaps < 1.2"; that explanation does not survive correction, because the damage is broad rather than marginal.

This is also evidence about the audit itself. The same measurement fix that *reduced* Experiment 6's claimed benefit *increased* Experiment 5's measured harm, so the defects were not biased toward flattering results.

Taken together, Experiments 12 and 13 sharpen the paper's central empirical claim. Training on 6 probes gives 2 fixed and 15 broken; training on 60 diverse probes gives 5 fixed and 0 broken on held-out prompts. The contrast between memorization and generalization is starker under sound measurement than under the original metric.

8 Errata and Corrections

Experiments 5 and 6 report inflated fix counts. The underlying finding—that ultra-sparse XOR patches produce targeted behavioral change with no measured collateral damage—replicates under corrected measurement. Version 1 results are annotated rather than rewritten, and the published artifact is preserved at git tag `v1.0`.

8.1 Origin

An independent reproduction in a standalone PyTorch Q1_0 engine reported two problems: the hardcoded wrong token was not the model's actual top competitor on 74 of 90 probes, and the integral category was a single token contrast repeated fifteen times rather than fifteen independent measurements. Both replicate here (76/90 and confirmed respectively). Auditing them surfaced a third, larger defect not previously reported. The reproduction also confirmed the baseline of 17/30 validation probes wrong, agreeing across MLX and GGUF/PyTorch runtimes.

8.2 Defect 1: Dead Probes

`encode_token()` keeps only the last subtoken of an answer string, and Bonsai's tokenizer splits digits into single characters. Both `" 20"` and `" 0"` therefore reduce to the same id, making correct_id = wrong_id and the logit gap *identically zero regardless of any weight flip*. Such a probe cannot be fixed, cannot be broken, and scores as wrong because its gap is not positive. Experiment 5 contains 7 such probes and Experiment 6 contains 8, four of them inside the 30-probe validation set. The reported baseline of "17 of 30 wrong" therefore included 4 probes incapable of being either right or wrong: the fixable pool was 13, not 17, so the reported 4/17 (23.5%) used an inflated denominator.

8.3 Defect 2: Measurement Offset

68 of 90 Experiment 6 answers are multi-token, so the scored id is not the token the model emits next; on 52 of 90 probes the model's true top-1 next token is a bare space, and the digit was scored one position early. Under `token_gap` the model appears to answer 13/30 validation probes correctly; scoring the full answer string gives 22/30 and greedy decoding gives 17/30.

8.4 Defect 3: Distractor Quality and Category Degeneracy

The hardcoded distractor was not the model's top competitor on 76/90 probes, median rank 23. Contrast diversity was uneven across categories: 10 distinct (correct, wrong) pairs for polynomial and exponential derivatives, 8 for second derivatives, 3 for trigonometry, 2 for primality, and **1** for integrals. The integral category is one measurement reported as fifteen. Primality is a two-token contrast but alternates direction across probes, so it cannot be won by a token bias and is not degenerate in the same way.

8.5 Corrected Result

Re-scoring the same 93-flip Experiment 6 patch under full-answer log-probability, changing nothing else, gives 2 held-out fixes rather than 4, with zero breakage. Three of the four published fixes (`pd_val_1`, `sd_val_3`, `int_val_4`) were already correct at baseline and were counted as failures only because of Defect 2; `int_val_4` is the integral probe from the degenerate category. The corrected metric additionally finds one fix the original missed (`pd_val_0`). Greedy decoding confirms both survivors: `d/dx [x^7 + x] =` changes from `0` to `7x^6 + 1`, and `Is 113 prime?` changes from `No` to `Yes`.

The claim of zero broken probes on held-out data holds under every metric tested. Experiment 5's sign-flip counts were affected by its 7 dead probes and understated the harm: rerun as Experiment 13 (Section 7), the 6-probe patch breaks 15 probes rather than 5. The defects were not biased toward flattering results—the same fix that reduced Experiment 6's claimed benefit increased Experiment 5's measured damage.

Correcting the objective did more than deflate the original number. Re-running the search itself against the sound metric (Experiment 12, Section 6), holding layers, iterations, seed and control probes fixed, yields **5** held-out fixes with **zero** breakage from a *smaller* patch—78 flips and 936 bytes against Experiment 6's 93 flips and 1,116 bytes—with every fix verified in free generation. The original search had spent its budget partly optimizing a mis-tokenized target scored against a rank-23 distractor; aiming it correctly recovered more than the defect had cost.

8.6 Methodological Change

Logit gaps are retained as a search signal but are no longer reported as a result. This implements the fix identified by Experiment 11's own interpretation—a more sensitive fitness function based on log-probability differences rather than logit gaps. The original metric remains available so Experiments 1–11 stay reproducible as published.

9 Limitations

The corrected metric is MLX-only. `seq_logprob` requires `Backend.seq_logprobs()`, implemented for MLX but not for GGUF: the `bankai_eval` subprocess exposes a single-gap `PROBE` command and would need a continuation-scoring command alongside it. Until then, Experiments 9–11 rest on `token_gap` and inherit its defects; their numbers warrant the same caution as Experiment 6's. The GGUF backend raises `NotImplementedError` rather than silently falling back.

Evaluation harness limitations. Our GSM8K accuracy (22%) is well below reported benchmarks, indicating evaluation setup differences. Logit gap probes don't always predict generation outcomes (visible in the 7×8 example). Standard benchmark evaluation with established harnesses is a next step.

Greedy search finds local optima. Population-based evolutionary search with crossover (XOR of XOR patches is a valid patch) could find better solutions.

Row-level granularity is the right operating point for mean-fitness search. Experiment 11 confirmed that naive per-group search fails: individual 128-bit flips produce probe-gap changes an order of magnitude smaller than row-level flips, and the control-degradation penalty dominates this tiny signal. Finer granularity requires a more sensitive fitness function or cumulative flipping (accepting groups of groups together).

Attention projections do not help generalization. Experiment 10 showed that XOR flips on attention Q/K/V/O are mechanically valid but encode context-specific routing patterns that overfit to training probes and regress on held-out ones. MLP is the right target for behavioral patching.

GGUF and MLX versions of Bonsai are not the same model. GGUF Q1_0_g128 is scale-only (1.125 bpw) while MLX is scale+bias (1.25 bpw), making GGUF strictly less expressive. Identical probe prompts produce different logit gaps on the two formats, and patches found on one do not necessarily transfer. Experiments 3–8 use MLX; 9–11 use GGUF.

Patch stacking shows interference. Experiment 7 confirms stacking is mechanically sound but behaviorally lossy—individual improvements partially cancel. Joint optimization would likely outperform naive stacking.

Single model. Bonsai 8B is currently the only production-quality true 1-bit LLM. The approach does not extend to ternary models.

Scale factors are not patched. Only binary weights are modified, not FP16 scale/bias values.

Small evaluation set. Our probes cover limited domains.

10 Responsible Use

Bankai modifies model behavior with kilobyte-scale patches invisible at inference time. The same mechanism could insert subtle malicious changes. However, XOR patches are transparent by design: every patch is a readable manifest listing exactly which rows were flipped, and verification is trivial. If patch libraries become a deployment pattern, provenance and verification become critical infrastructure.

11 Future Work

- **Finer granularity:** Group-level (128 bits) or per-bit search for more precise patches.
- **Evolutionary search:** Population-based with crossover via XOR and Hamming-distance diversity pressure.
- **Benchmark evaluation:** MMLU subcategories, GSM8K, HumanEval.
- **Patch stacking:** Empirical composability testing.
- **Cross-model extraction:** XOR between Bonsai variants as naturally occurring patches.
- **Calculus-specific layer selection:** Searching layers 5, 6, and 10 (identified as high-impact for calculus).

12 Conclusion

We have introduced Bankai, the first post-training adaptation method for true 1-bit LLMs. By exploiting the binary nature of weights, behavioral modifications reduce to sparse XOR bitmasks that are orders of magnitude smaller than existing adapters, incur zero inference overhead, and can be applied and reverted in microseconds.

Through eleven experiments on Bonsai 8B, we characterized the method's capabilities and limitations: targeted corrections work reliably, generalization requires diverse training signal, row-level MLP flips are the right operating point (finer granularity fails at naive greedy search; attention projections overfit), and the approach ports to NVIDIA GPUs via a custom llama.cpp tool that runs ∼24× faster than the Apple Silicon baseline. The method represents a new point in the adaptation design space—one that is uniquely enabled by true binary model architectures.

Code, patches, and all experiments are available at https://github.com/nikshepsvn/bankai.

References

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. *arXiv:2106.09685*, 2021.

A. S. Rakin, Z. He, and D. Fan. Bit-flip attack: Crushing neural network with progressive bit search. In *ICCV*, 2019.

P. Dong, et al. STBLLM: Breaking the 1-bit barrier with structured binary LLMs. In *ICLR*, 2025.

H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei. BitNet: Scaling 1-bit transformers for large language models. *arXiv:2310.11453*, 2023.

S. Ma, H. Wang, S. Huang, et al. BitNet b1.58 2B4T technical report. *arXiv:2504.12285*, 2025.

PrismML. Bonsai 8B. https://prismml.com/news/bonsai-8b, 2026.

K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in GPT. In *NeurIPS*, 2022.

K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. In *ICLR*, 2023.

Y. Bai, et al. Targeted attack against deep neural networks via flipping limited weight bits. In *ICLR*, 2021.

Y. Xu, et al. RECAST: Reparameterized, compact weight adaptation for sequential tasks. *arXiv:2411.16870*, 2024.

A. Bulat and G. Tzimiropoulos. XOR-Net: An efficient computation pipeline for binary neural network inference on edge devices. 2020.

M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *ECCV*, 2016.

T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. In *NeurIPS*, 2023.